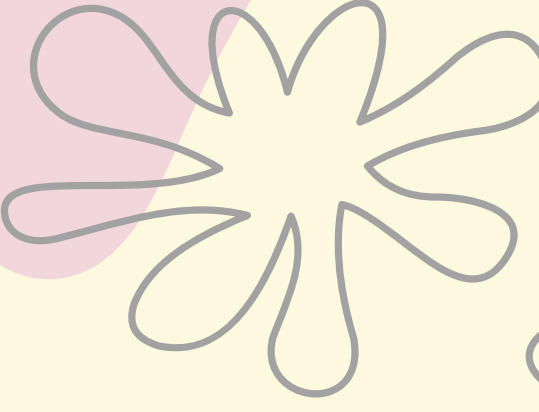
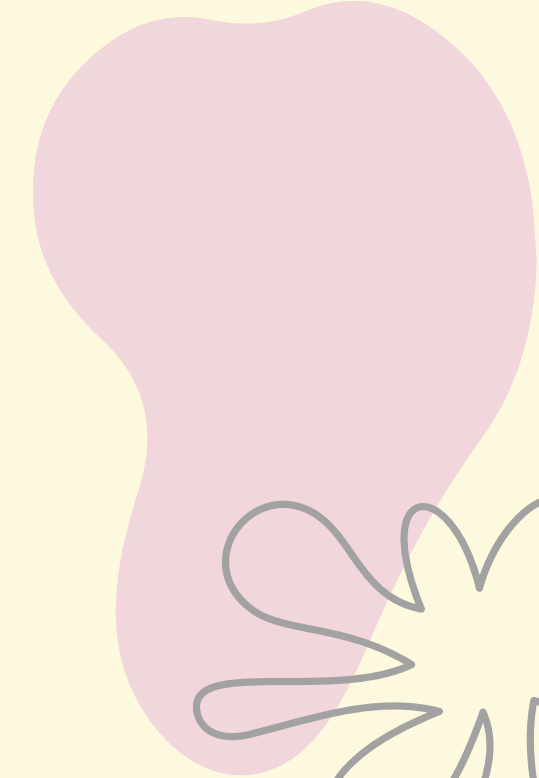
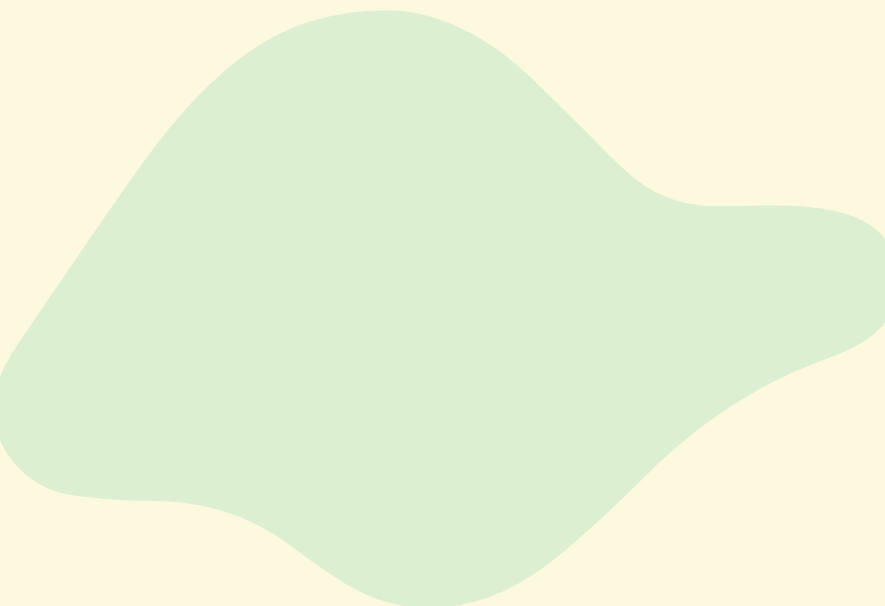




DISASTER RESPONSE LOGISTICS SYSTEM

kelompok 4



Anggota Kelompok



Noomira Zakiah
(25051030072)



M. Farras Ashiddiq
(25051030064)
Ketua Team



Fahri Khairu Imtihan
(25051030052)



Restu Pamungkas
(25051030080)

Latar Belakang

Indonesia adalah salah satu negara dengan risiko bencana alam tertinggi di dunia (gempa bumi, banjir, erupsi gunung berapi). Koordinasi logistik tanggap bencana yang cepat dapat menyelamatkan ribuan nyawa. Sistem logistik berbasis struktur data memungkinkan: pengiriman bantuan ke lokasi KRITIS lebih cepat (Priority Queue), pencarian rute distribusi optimal dari depot ke lokasi terdampak (Dijkstra), registrasi dan pembaruan lokasi bencana secara cepat (BST), dan audit seluruh pengiriman (Stack). Proyek ini mensimulasikan sistem logistik untuk 35 lokasi terdampak dan 3 depot logistik.

Distribusi bantuan bencana membutuhkan sistem yang cepat dan efisien Indonesia terletak di Ring of Fire, menjadikannya salah satu wilayah paling rawan bencana di dunia.

- Prioritas: Kebutuhan memisahkan bantuan KRITIS vs RINGAN.
- Optimasi: Penentuan rute tercepat di tengah kerusakan infrastruktur.
- Monitoring: Pencatatan log transaksi yang akurat.

Tujuan Sistem

1. Mengelola data ribuan desa terdampak secara terstruktur untuk pencarian dan pembaruan status yang cepat.
2. Mengimplementasikan algoritma jalur terpendek untuk memastikan bantuan sampai melalui rute yang paling efisien.
3. Memastikan wilayah dengan kategori "KRITIS" mendapatkan bantuan lebih dulu dibandingkan wilayah "SEDANG" atau "RINGAN".
4. Mengidentifikasi secara dini area yang tidak dapat dijangkau oleh depot pusat akibat infrastruktur yang lumpuh (jalur terputus).
5. Menyediakan log riwayat pengiriman yang akurat dengan kemampuan rollback untuk menjaga akurasi stok bantuan di gudang.

Konsep Sistem

1. Konsep Database (BST Registry): Menggunakan Binary Search Tree (BST). Data setiap lokasi disimpan dengan kunci kode_lokasi.
2. Konsep Triage (Priority Queue): Menggunakan antrean prioritas berbasis Linked List. Bantuan tidak diproses berdasarkan waktu masuk (First-In-First-Out), melainkan berdasarkan bobot urgensi. Konsep ini memastikan "Level 1 (Kritis)" selalu berada di urutan terdepan antrean armada.
3. Konsep Pemetaan (Graph Network): Wilayah bencana direpresentasikan sebagai Graph. Posko dan desa dianggap sebagai Node, sedangkan akses jalan dianggap sebagai Edge dengan bobot berupa jarak (km).
4. Konsep Riwayat & Rollback (Stack): Menggunakan konsep Last-In-First-Out (LIFO). Setiap transaksi yang berhasil diproses akan "ditumpuk" ke dalam Stack. Jika terjadi kesalahan input atau pembatalan pengiriman terakhir, sistem akan melakukan Pop untuk mengembalikan status bantuan.

STRUKTUR DATA

1. Graph Jaringan Rute

Graph jaringan rute merupakan graf berbobot tidak berarah yang digunakan untuk menyimpan hubungan antar lokasi beserta jaraknya dalam kilometer. Data disimpan menggunakan adjacency list berbasis Linked List sehingga pencarian tetangga lebih efisien. Sistem mendukung operasi tambah_node, tambah_rute, tetangga, dan BFS dari depot untuk mendeteksi lokasi yang tidak terjangkau. Kompleksitas penambahan data adalah $O(1)$, sedangkan BFS memiliki kompleksitas $O(V+E)$.

5. Dijkstra & Audit Jarak

Dijkstra dan Audit Jarak digunakan untuk mencari rute dengan jarak minimum dari setiap depot ke seluruh lokasi. Setelah jarak diperoleh, data lokasi diurutkan menggunakan Selection Sort pada Linked List berdasarkan jarak ke depot terdekat untuk mengidentifikasi lokasi yang paling sulit dijangkau. Kompleksitas proses ini adalah $O(V^2+E)$.

3. BST Registry Lokasi

BST Registry Lokasi digunakan untuk menyimpan data lokasi berdasarkan kode_lokasi sebagai kunci utama. Setiap node menyimpan informasi seperti kode, nama lokasi, level bencana, populasi, dan status. Sistem mendukung operasi insert, search, update_level, serta inorder untuk menampilkan data secara terurut. Kompleksitas rata-rata setiap operasi pada BST adalah $O(\log n)$.

2. Priority Queue Bantuan

Priority Queue bantuan digunakan untuk mengatur antrian pengiriman bantuan berdasarkan tingkat prioritas bencana. Struktur ini menggunakan Linked List sehingga lokasi dengan level KRITIS (level=1) selalu diproses lebih dahulu. Sistem mendukung operasi KIRIM untuk menambahkan bantuan sesuai prioritas dan PROSES_BANTUAN untuk mengambil serta mengirim bantuan dari antrian. Kompleksitas enqueue adalah $O(n)$, sedangkan dequeue memiliki kompleksitas $O(1)$.

4. Stack Log Pengiriman

Stack Log Pengiriman digunakan untuk menyimpan riwayat transaksi pengiriman bantuan dengan metode LIFO (Last In First Out). Struktur ini mendukung operasi LOG_PENGIRIMAN untuk menampilkan riwayat pengiriman dan ROLLBACK untuk membatalkan pengiriman terakhir serta mengembalikan stok ke depot. Proses push dan pop pada stack memiliki kompleksitas $O(1)$.

KEPUTUSAN DESIGN

MODUL	STRUKTUR DATA	ALASAN MEMILIH
GRAPH RUTE	Adjacency list	Dipakai untuk menyimpan peta rute antar lokasi. Dipilih karena koneksi antar lokasi tidak padat (hanya sedikit yang terhubung langsung), jadi lebih hemat memori daripada menyimpan semua kemungkinan rute dalam sebuah tabel besar. Rute juga bisa dilalui dua arah, seperti jalan pada umumnya
PRIORITY QUEUE BANTUAN	Sorted linked list	Dipakai untuk mengantre bantuan yang akan dikirim. Dipilih agar lokasi yang paling parah (KRITIS) selalu dilayani lebih dulu, bukan yang duluan masuk antrean. Caranya dengan menyusun antrean berdasarkan tingkat keparahan sejak awal, sehingga pengambilan berikutnya selalu yang paling mendesak.
BST LOKASI	Binary Search Tree	Dipakai untuk menyimpan data semua lokasi (nama, level bencana, populasi). Dipilih karena bisa mencari lokasi dengan cepat berdasarkan kode, sekaligus bisa menampilkan daftar lokasi secara urut alfabetis secara otomatis – berguna untuk laporan bencana.
STACK LOG PENGIRIMAN	Stack (LIFO)	Dipakai untuk mencatat riwayat pengiriman bantuan. Dipilih karena fitur ROLLBACK (batalkan pengiriman) selalu membatalkan yang paling terakhir dilakukan – persis seperti tombol "undo" di aplikasi. Struktur Stack secara alami bekerja dengan prinsip "yang terakhir masuk, yang pertama keluar".
DJIKSTRA	Algoritma Dijkstra	Dipakai untuk mencari rute terpendek dari depot ke lokasi tujuan. Dipilih karena semua jarak antar lokasi bernilai positif (tidak ada jarak minus), dan jumlah lokasi sedikit sehingga cara ini sudah cukup cepat. Hasilnya bisa langsung menampilkan urutan lokasi yang harus dilalui.

TAMPILAN OUTPUT

```
PS C:\Users\DELL\OneDrive - uny.ac.id\Dokumen\GitHub\tpb-asd-Disaster-Response-Logistics-System-kel04> & C:\msys64\ucrt64\bin\python.exe "c:/Users/DELL/OneDrive
uny.ac.id/Dokumen/GitHub/tpb-asd-Disaster-Response-Logistics-System-kel04/tpb-asd-kelompok-04/src/main.py"
```

Disaster Response Logistics System. Ketik BANTUAN untuk daftar perintah.

Logistik-CLI> bantuan

Daftar Perintah Tersedia:

KIRIM <depot> <lokasi> <jenis_bantuan> <jumlah>

PROSES_BANTUAN

RUTE_OPTIMAL <depot> <tujuan>

UPDATE_LEVEL <kode_lokasi> <level_baru>

TIDAK_TERJANGKAU <depot>

LOG_PENGIRIMAN

LAPORAN_BENCANA

ROLLBACK

KELUAR

Logistik-CLI> laporan_bencana

KODE	NAMA LOKASI	LEVEL	POPULASI
DEPOT_0	Gudang Logistik 0	RINGAN	0
DEPOT_1	Gudang Logistik 1	RINGAN	0
DEPOT_2	Gudang Logistik 2	RINGAN	0
L000	Desa/Kelurahan-0	RINGAN	2764
L001	Desa/Kelurahan-1	SEDANG	3792
L002	Desa/Kelurahan-2	KRITIS	485
L003	Desa/Kelurahan-3	SEDANG	4837
L004	Desa/Kelurahan-4	SEDANG	2196
L005	Desa/Kelurahan-5	SEDANG	4303
L006	Desa/Kelurahan-6	KRITIS	1533
L007	Desa/Kelurahan-7	RINGAN	3777
L008	Desa/Kelurahan-8	SEDANG	4154
L009	Desa/Kelurahan-9	RINGAN	4126
L010	Desa/Kelurahan-10	SEDANG	1513
L011	Desa/Kelurahan-11	RINGAN	1620
L012	Desa/Kelurahan-12	RINGAN	2797
L013	Desa/Kelurahan-13	RINGAN	1048
L014	Desa/Kelurahan-14	KRITIS	2077
L015	Desa/Kelurahan-15	KRITIS	2800
L016	Desa/Kelurahan-16	RINGAN	4796
L017	Desa/Kelurahan-17	RINGAN	4775
L018	Desa/Kelurahan-18	RINGAN	4305
L019	Desa/Kelurahan-19	RINGAN	3682
L020	Desa/Kelurahan-20	SEDANG	2767
L021	Desa/Kelurahan-21	SEDANG	1526

TAMPILAN OUTPUT

```
L022      | Desa/Kelurahan-22      | RINGAN | 4373
L023      | Desa/Kelurahan-23      | RINGAN | 4973
L024      | Desa/Kelurahan-24      | RINGAN | 2474
L025      | Desa/Kelurahan-25      | RINGAN | 2319
L026      | Desa/Kelurahan-26      | RINGAN | 2724
L027      | Desa/Kelurahan-27      | RINGAN | 4715
L028      | Desa/Kelurahan-28      | KRITIS | 510
L029      | Desa/Kelurahan-29      | SEDANG | 3187
L030      | Desa/Kelurahan-30      | KRITIS | 932
L031      | Desa/Kelurahan-31      | RINGAN | 348
L032      | Desa/Kelurahan-32      | SEDANG | 1526
L033      | Desa/Kelurahan-33      | RINGAN | 561
L034      | Desa/Kelurahan-34      | SEDANG | 191

Logistik-CLI> kirim DEPOT_0 L001 makanan 500
Sukses: Bantuan ID 1 (Prioritas: 2) dimasukkan ke antrian.

Logistik-CLI> kirim DEPOT_0 L000 obat 200
Sukses: Bantuan ID 2 (Prioritas: 3) dimasukkan ke antrian.

Logistik-CLI> proses_bantuan
[PENGIRIMAN] Memproses Bantuan ID 1:
  Kirim 500 x MAKANAN dari DEPOT_0 ke L001

Logistik-CLI> proses_bantuan
[PENGIRIMAN] Memproses Bantuan ID 2:
  Kirim 200 x OBAT dari DEPOT_0 ke L000

Logistik-CLI> rute_optimal DEPOT_0 L000
Rute Terpendek (290 km): DEPOT_0 -> L012 -> L013 -> L025 -> L006 -> L000

Logistik-CLI> rute_optimal DEPOT_0 L001
Perintah tidak dikenali. Ketik BANTUAN untuk melihat daftar perintah.

Logistik-CLI> rute_optimal DEPOT_0 L000
Rute Terpendek (290 km): DEPOT_0 -> L012 -> L013 -> L025 -> L006 -> L000

Logistik-CLI> log_pengiriman
=== RIWAYAT TRANSAKSI PENGIRIMAN (LIFO) ===
ID 2: 200 OBAT (DEPOT_0 -> L000)
ID 1: 500 MAKANAN (DEPOT_0 -> L001)

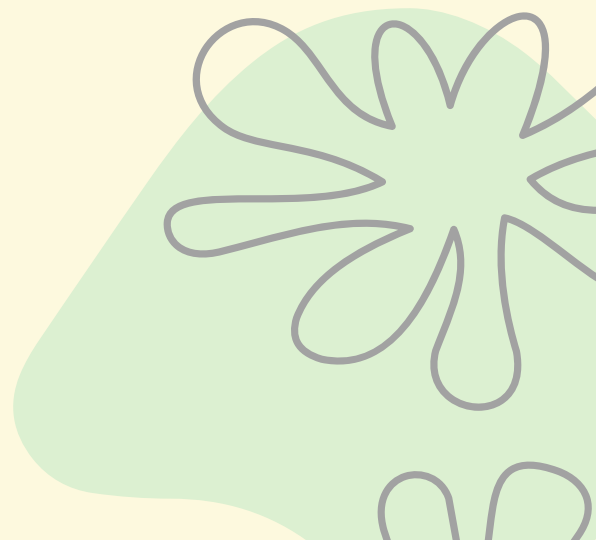
Logistik-CLI> keluar
Keluar dari sistem. Terima kasih.
```

MODUL	OPERASI	BIG-O	EFISIENS I	PENJELASAN
GRAPH	Tambah node / rute	$O(1)$	Sangat cepat	Langsung tempel di depan linked list, tidak perlu geser elemen lain.
	Lihat tetangga	$O(\text{deg})$	Sangat cepat	Hanya scan tetangga yang benar-benar terhubung, bukan semua lokasi.
PRIORITY QUEUE	Enqueue	$O(n)$	Cukup	Perlu cari posisi yang tepat di antrean. Masih oke karena jumlah antrian kecil.
	Dequeue	$O(1)$	Sangat cepat	Langsung ambil dari kepala antrean yang paling kritis selalu ada di depan.
BST lokasi	Insert lokasi baru	$O(\log n)$	Cepat	Setiap langkah memangkas setengah kemungkinan – seperti tebak angka biner.
	Cari / update level	$O(\log n)$	Cepat	Dengan 35 lokasi, hanya butuh sekitar 5–6 langkah untuk menemukan lokasi.
Stack log	Push (catat kirim)	$O(1)$	Sangat Cepat	Taruh di atas tumpukan – tidak perlu geser apapun.
Dijkstra	Cari rute terpendek	$O(V^2+E)$		Dengan hanya 38 lokasi, ini setara ~1.400 operasi – selesai dalam sekejap.

KESIMPULAN

Proyek Disaster Response Logistics System merupakan sistem simulasi distribusi bantuan bencana berbasis CLI yang membuktikan bahwa pemilihan struktur data yang tepat adalah kunci efektivitas sistem logistik darurat. Sistem ini mengintegrasikan Graph Adjacency List untuk memetakan jaringan rute antar 35 lokasi dan 3 depot, Priority Queue berbasis Sorted Linked List untuk menjamin lokasi KRITIS selalu dilayani lebih dulu, Binary Search Tree untuk pencarian data lokasi cepat $O(\log n)$ sekaligus menghasilkan laporan terurut otomatis, Stack LIFO untuk audit riwayat pengiriman dan fitur ROLLBACK, serta Algoritma Dijkstra untuk menentukan rute terpendek berdasarkan bobot jarak seluruhnya dibangun manual tanpa library bawaan Python.

Meskipun terdapat trade-off yang disadari seperti enqueue $O(n)$ pada Priority Queue, keterbatasan Adjacency List dibanding matrix, dan Dijkstra tanpa heap yang kurang optimal di skala besar, semua keputusan desain tersebut tetap memadai untuk skala simulasi ini dan masih terbuka untuk dikembangkan lebih lanjut menggunakan Min-Heap, Fibonacci Heap, maupun Dynamic Graph menuju sistem penanganan bencana berbasis peta geografis nyata secara real-time.



**Thank
You**